ZELORA

SALON MANAGEMENT SYSTEM

A desktop application designed with Object-Oriented Programming

ICT 1306 — OBJECT ORIENTED PROGRAMMING

Group Assignment Report

Department of Computing
Rajarata University of Sri Lanka

Project Scenario	**Salon Management System**
Application Name	**Salon ZELORA**
Technology	**C++ / C++/CLI** **Visual Studio Windows Forms** **MySQL**
Team Members	**R.G.T.D.Pushpakumara ICT/2024/102** **P.K.C.N.Nonis ICT/2024/097** **W.N.Kavindya ICT/2024/071** **A.G.A.Anha ICT/2024/012**
GitHub Repository	**https://github.com/Tharushi-2004/ZELORA**
Demo Video	https://youtu.be/Am-5Fx7w2mE

"Elegant salon operations. Simple digital management."

Project Overview

Purpose, scope and core system features

ZELORA is a desktop-based Salon Management System developed to simplify the day-to-day activities of a salon. The application provides a centralized interface for managing customers, salon services, staff members and appointments while storing records in a MySQL database. The project was designed to demonstrate practical use of Object-Oriented Programming concepts together with a graphical user interface and database connectivity.

Main Objectives

- Provide an easy-to-use Windows Forms interface for salon administration.
- Maintain customer, service, staff and appointment information in an organized way.
- Support CRUD operations: Create, Read, Update and Delete records.
- Apply class, object, encapsulation, abstraction, inheritance and polymorphism in C++.
- Connect the desktop application to a real MySQL database.

Application Modules

Module	Purpose	Typical Operation
Login	Controls entry to the system	Validate admin credentials
Customer Management	Stores client details	Add / view / update / delete
Service Management	Stores salon services and prices	Maintain service records
Staff Management	Stores staff information	Maintain staff records
Appointment Management	Organizes salon bookings	Create and manage appointments
Dashboard	Central navigation screen	Open management modules

Technology Stack

Front End	Visual Studio Windows Forms using C++/CLI
Programming	C++ with Object-Oriented Programming
Database	MySQL Server / MySQL Workbench
Connector	MySQL Connector/C++
Version Control	Git and GitHub

02 CLASS DESIGN

Class List & Responsibilities

Core classes used to model the salon domain

Class	Responsibility	Key Data / Behaviour
Person	Base class for people in the system.	Name, phone, email; common person behaviour.
Customer	Represents salon customers.	Customer-specific details; inherits Person.
Staff	Represents salon employees.	Staff details/role; inherits Person.
Service	Represents a salon service.	Service ID, name, price and service information.
Appointment	Represents a customer booking.	Customer, service, date, time and appointment details.
Database	Handles communication with MySQL.	Connection setup and database operations.

GUI Forms

- LoginForm — authentication screen.
- DashboardForm — main navigation hub.
- CustomerForm — customer record management.
- ServiceForm — salon service management.
- AppointmentForm / BookAppointmentForm — booking management.
- StaffForm — staff record management.
- AboutForm — application information.

Design Rationale

The class structure separates data and responsibilities into meaningful objects. Common person information is placed in a reusable base class, while Customer and Staff represent specialized people. Service and Appointment model the salon's operational data, and Database isolates database-related work from the user-interface forms. This makes the program easier to understand, maintain and extend.

OOP Concepts — Part I

Worked examples based on the ZELORA design

1. Class

A class is a blueprint that groups data and related behaviour. In ZELORA, Service is used to represent a salon service.

```
class Service {
private:
    int serviceId;
    string serviceName;
    double price;
public:
    Service(int id, string name, double p);
    void displayInfo();
};
```

2. Object

An object is an instance created from a class. For example, a haircut service can be represented as an object of Service.

```
Service haircut(1, "Hair Cut", 2500.00);
haircut.displayInfo();
```

3. Encapsulation

Encapsulation protects internal data by keeping attributes private and allowing controlled access through public methods. This prevents other parts of the program from directly changing sensitive object data.

```
class Person {
private:
    string name;
    string phone;
public:
    void setName(string n) { name = n; }
    string getName() const { return name; }
};
```

Why it matters in ZELORA

- Customer and staff information can be validated before values are changed.
- The internal representation of a class can be changed without rewriting the entire application.
- Forms interact with objects through clear public methods instead of uncontrolled direct access.

OOP Concepts — Part II

Abstraction, inheritance and polymorphism

4. Abstraction

Abstraction exposes only the necessary operation while hiding implementation details. A form can request a database connection without needing to know every low-level connector step.

```
class Database {
public:
    bool connect();
    void disconnect();
    // Connection details are handled internally.
};
```

5. Inheritance

Inheritance allows a new class to reuse the properties and behaviour of an existing class. Customer and Staff can share common information from Person.

```
class Person {
protected:
    string name;
    string phone;
public:
    virtual void displayInfo();
};

class Customer : public Person {
private:
    int customerId;
};

class Staff : public Person {
private:
    int staffId;
};
```

6. Polymorphism

Polymorphism allows the same method call to behave differently depending on the object. A virtual displayInfo() function can be overridden by derived classes.

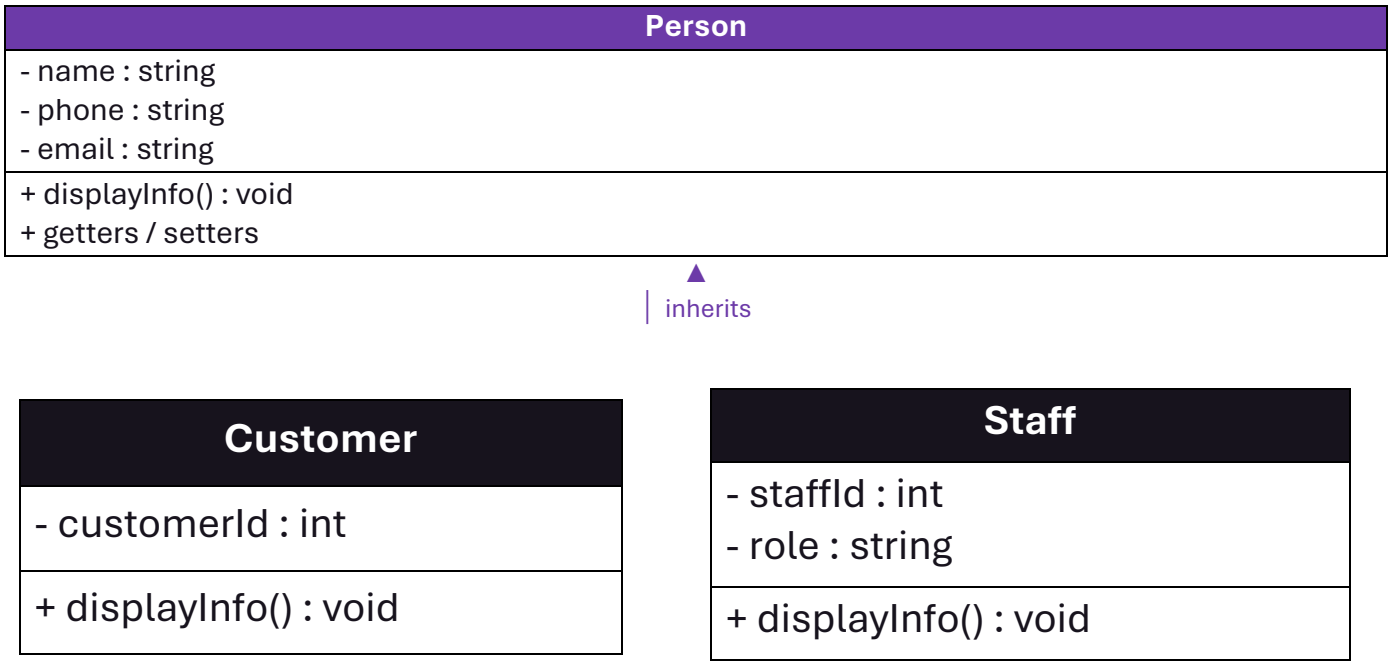
```
class Customer : public Person {
public:
    void displayInfo() override {
        // Display customer information
    }
};

class Staff : public Person {
public:
    void displayInfo() override {
        // Display staff information
    }
};
```

05 CLASS DIAGRAM

Class Diagram

Simplified conceptual view of the main ZELORA classes



Relationship summary: Customer and Staff specialize Person. Appointment connects salon booking information with customer and service data. Database supports persistent storage for the application's management modules.

06 DATABASE & GUI

Database Integration & User Interface

How the desktop application manages real salon records

Database Integration

ZELORA uses MySQL as the persistent data store and MySQL Connector/C++ to communicate with the database from the Visual Studio application. The management screens are designed to perform full CRUD operations on salon records.

CRUD	Meaning	ZELORA Example	Result
Create	Add	Register a new customer/service/appointment	New row stored
Read	View	Load records to the DataGridView	Existing data displayed
Update	Edit	Change selected record information	Stored row updated
Delete	Remove	Delete a selected record	Row removed

GUI Flow

LOGIN	DASHBOARD	CUSTOMERS	SERVICES	APPOINTMENTS

Login → Dashboard → Select a management module → Perform record operations → Return to Dashboard / Logout

Key Interface Features

- Multiple Windows Forms screens with clear navigation.
- Text fields and buttons for entering and managing salon information.
- DataGridView-based record display for management screens.
- Dashboard buttons for Customers, Services, Appointments, Staff and Logout.
- Simple administrator login to control access to the application.

Conclusion & Submission Links

Project outcome and final checklist

Conclusion

The ZELORA Salon Management System demonstrates how Object-Oriented Programming can be applied to a realistic small-business scenario. The project combines reusable C++ classes, a Visual Studio Windows Forms interface and a MySQL database to manage important salon operations. By separating responsibilities among Person, Customer, Staff, Service, Appointment and Database classes, the application is structured in a way that supports maintainability and future expansion.

Project Highlights

- Practical salon management scenario with a branded desktop interface.
- Six core OOP concepts demonstrated through the application design.
- More than three GUI screens implemented in Visual Studio.
- MySQL-backed record management with CRUD functionality.
- Git-based source control and public GitHub repository.

Submission Links

GitHub	https://github.com/Tharushi-2004/ZELORA
5-minute Demo Video	https://youtu.be/Am-5Fx7w2mE

Final Checklist

Team of 5 and scenario details	☐ **Add final names + index numbers**
2+ classes and all 6 OOP concepts	✓ **Covered**
3+ GUI screens	✓ **Covered**
MySQL with CRUD operations	✓ **Covered**
Class diagram	✓ **Included**
Public GitHub URL	✓ **Included**
5-minute unlisted demo video	☐ **Add final YouTube link**